

Øvelser: sentimentanalyse med asent

Social Data Science 1: lektion 7

Øvelserne bruger asent fra slides (*asent: sentimentanalyse*) på det datasæt, vi har preprocessed sammen, `ft_lovforslag.csv`. Slides viste, hvad asent kan, som en ordbog ikke kan. Her undersøger vi, hvor godt den passer til Folketingets sprog:

1. Sætninger, der ligner Folketinget: høflighed, faste vendinger og nægtelser.
2. Tonen i debatterne, og om den stemmer med ordbogen fra slides.
3. Hvilke ord der driver tonen.
4. Tonen sætning for sætning, og en minimumsvalidering.

Under **Hjælp** står, hvor I har set de funktioner, I skal bruge:

- **Slides:** lektion 7-slides, med navnet på slidet.
 - **Preprocess-notebooken:** `l7_preprocess_dkparl_trinvis`, med trinnets nummer.
 - **spaCy-notebooken:** `l7_spacy_supplement`, med trinnets nummer.
-

Opsætning

asent kører oven på spaCy og kan ikke køre i browseren. spaCy-modellen og asent installeres én gang i terminalen, som i spaCy-notebooken, trin 1:

```
pip install spacy asent
python -m spacy download da_core_news_sm
```

Kør derefter cellen herunder. Ret stien, så den peger på jeres egen kopi af `ft_lovforslag.csv`.

```
!python -m spacy download da_core_news_sm
```

```
import pandas as pd
import numpy as np

sti = "/Users/jeppefl/Library/CloudStorage/OneDrive-
AalborgUniversitet/01_work/01_undervisning/02_sds1/01_slides/lektion-7/
resources/ft_lovforslag.csv"
korpus = pd.read_csv(sti)

print(korpus.shape)
```

```
(3070, 9)
```

spaCy er langsom på hele materialet (spaCy-notebooken, trin 3.2). Øvelse 2-4 bruger derfor det samme udsnit på 100 debatter som spaCy-notebooken, trin 3.1.

1. asent på Folketingets sprog

Slides viste, at asent kan se negation, som vores egen ordbog ikke kunne (*Hvad den kan, som vores egen ikke kunne*). Men slides testede sætninger, der ligner nyhedstekst. Her tester vi sætninger, der ligner Folketinget: høflighedsformler, *godt* i faste vendinger og nægtelser med dansk ordstilling.

Opgave:

1. Lav pipelinen fra slides: `da_core_news_sm` med `sentencizer` og `asent_da_v1`.
2. Kør sætningerne herunder igennem, og lav en tabel med sætningen, dens compound og de ord, der drev scoren.

```
test = pd.Series([
    "Det er en god beslutning for vores frihed.",
    "Det er ikke en god beslutning.",
    "Det er godt, at ministeren er her.",
    "Godt nok er forslaget dyrt, men vi støtter det.",
    "Vi kan godt tænke os at se på det.",
    "Tak for ordet, og tak til ministeren for et godt samarbejde.",
    "Vi støtter ikke forslaget.",
    "Vi kan ikke støtte forslaget.",
    "Regeringen svigter de ældre.",
])
```

Hjælp:

- Slides, `asent: sentimentanalyse` `nlp.add_pipe("sentencizer")`, `nlp.add_pipe("asent_da_v1")` og `doc._.polarity`.
- Slides, *Hvorfor fik den den score?:* `tok._.polarity.polarity` for hvert token.
- spaCy-notebooken, trin 2: en lille funktion, der bruges på hvert Doc. Her kan den returnere en liste med de ord, der har en polaritet.
- `.map(nlp)` sender hver sætning gennem pipelinen og giver en Series med ét Doc per sætning.

```
# Din kode her
import spacy
import asent

nlp = spacy.load("da_core_news_sm")
nlp.add_pipe("sentencizer")
nlp.add_pipe("asent_da_v1")

print(nlp.pipe_names)
```

```
/Users/jeppefl/Library/Python/3.9/lib/python/site-packages/urllib3/
__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+,
currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://
github.com/urllib3/urllib3/issues/3020
warnings.warn(
```

```
['tok2vec', 'morphologizer', 'parser', 'lemmatizer', 'attribute_ruler', 'ner',
'sentencizer', 'asent_da_v1']
```

```
def drivere(doc):
    return [(t.text, round(t._polarity.polarity, 2)) for t in doc if
t._polarity.polarity != 0]
```

```
test = pd.Series([
    "Det er en god beslutning for vores frihed.",
    "Det er ikke en god beslutning.",
    "Det er godt, at ministeren er her.",
    "Godt nok er forslaget dyrt, men vi støtter det.",
    "Vi kan godt tænke os at se på det.",
    "Tak for ordet, og tak til ministeren for et godt samarbejde.",
    "Vi støtter ikke forslaget.",
    "Vi kan ikke støtte forslaget.",
    "Regeringen svigter de ældre.",
])

docs = test.map(nlp)

mikro = pd.DataFrame({
    "saetning": test,
    "compound": [d._polarity.compound for d in docs],
    "drivere": docs.map(drivere),
}).round(3)

print(mikro.to_string())
```

	saetning	compound
drivere		
0	Det er en god beslutning for vores frihed.	0.791
	[(god, 3.0), (frihed, 2.0)]	
1	Det er ikke en god beslutning.	-0.497
	[(god, -2.22)]	
2	Det er godt, at ministeren er her.	0.612
	[(godt, 3.0)]	
3	Godt nok er forslaget dyrt, men vi støtter det.	0.840

4	[(Godt, 3.0), (dyrt, 3.0), (støtter, 2.0)]	
	Vi kan godt tænke os at se på det.	0.612
5	[(godt, 3.0)]	
	Tak for ordet, og tak til ministeren for et godt samarbejde.	0.875
6	[(Tak, 2.0), (tak, 2.0), (godt, 3.0)]	
	Vi støtter ikke forslaget.	0.459
7	[(støtter, 2.0)]	
	Vi kan ikke støtte forslaget.	-0.357
8	[(støtte, -1.48)]	
	Regeringen svigter de ældre.	-0.459
	[(svigter, -2.0)]	

Kig efter i viewer: Åbn tabellen. Hvilke scorer ville I være uenige i?

Til diskussion:

- Sammenlign *Vi støtter ikke forslaget* og *Vi kan ikke støtte forslaget*. Hvad er forskellen på de to sætninger, og hvorfor giver asent dem modsat fortegn?
- Hvordan behandler asent *godt* i *godt nok* og *kan godt tænke os*? Er det bedre end ordbogen fra slides?
- Er der ord, der får en polaritet, I ikke havde forventet?

Jeres svar:

2. Tonen i debatterne, og stemmer den med ordbogen?

Slides kørte asent på debatterne og fandt, at Folketinget er høfligt (*Folketinget er høfligt* og *Per politikområde*). Her gør vi det samme på et udsnit og tjekker, om asent og ordbogen fra slides er enige om, hvilke debatter der er positive.

Opgave:

1. Tag et udsnit på 100 debatter med `random_state=1`, som i spaCy-notebooken, trin 3.1, og kørs teksterne gennem nlp. Tag tid på det.
2. Gem compound for hver debat i kolonnen `sentiment`.
3. Beskriv fordelingen. Hvor stor en andel af debatterne er negative?
4. Lav en oversigt med antal debatter og gennemsnitlig sentiment per politikområde.
5. Beregn ordbogens score for de samme debatter med ordbogen fra slides (*Vi bygger én*), både med og uden *godt*. Gem dem som `score` og `uden_godt`.
6. Beregn rangkorrelationen (Spearman) mellem `sentiment`, `score` og `uden_godt`.

```
ordbog = {"god": 2, "godt": 2, "bedre": 2, "bedst": 3, "stærk": 2,
          "sand": 2, "sandhed": 2, "frihed": 3, "tryghed": 2, "stolt": 2,
          "dårlig": -2, "værre": -2, "vørst": -3, "katastrofe": -3,
          "løgn": -3, "svigt": -2, "trussel": -2, "farlig": -2,
          "krise": -2, "vold": -3, "frygt": -2, "svag": -2}
```

Hjælp:

- spaCy-notebooken, trin 3.1 og 3.2: `.sample(100, random_state=1)`, `nlp.pipe(..., batch_size=50)` og `time.perf_counter()`.
- Slides, *Kør den på korpusset* (asent): `[doc._.polarity.compound for doc in ...]`.
- Preprocess-notebooken, trin 5.2: `.agg(antal="size", ...)`.
- Slides, *Hvordan den scorer* og *Kør den på korpusset*: tokenisering, `.map(ordbog)` og `.groupby(level=0).sum().where(ord != "godt")` sætter *godt* til NaN, så det ikke tæller med.
- `.corr(method="spearman")` giver rangkorrelationen mellem kolonnerne i en tabel.

```
# Din kode her
import time

udsnit = korpus.sample(100, random_state=1).copy()

start = time.perf_counter()
udsnit["doc"] = list(nlp.pipe(udsnit["tekst"], batch_size=50))
print("Tid:", round(time.perf_counter() - start, 1), "sekunder")
```

Tid: 147.4 sekunder

```
udsnit["sentiment"] = [doc._.polarity.compound for doc in udsnit["doc"]]

print(udsnit["sentiment"].describe().round(3))
print()
print("Andel negative:", round((udsnit["sentiment"] < 0).mean(), 3))
```

```
count    100.000
mean      0.144
std       0.071
min      -0.023
25%       0.098
50%       0.141
75%       0.186
max       0.315
Name: sentiment, dtype: float64
```

Andel negative: 0.03

```
print(udsnit.groupby("omraade")["sentiment"]
      .agg(antal="size", gennemsnit="mean")
      .sort_values("gennemsnit")
      .round(3))
```

	antal	gennemsnit
omraade		
Retsvæsen	11	0.077
Udlændinge	5	0.103
Skat	11	0.116
Beskæftigelse	10	0.156
Andet	47	0.157
Miljø og klima	10	0.174
Uddannelse	6	0.181

```
ordbog = {"godt": 2, "godt": 2, "bedre": 2, "bedst": 3, "stærk": 2,
          "sand": 2, "sandhed": 2, "frihed": 3, "tryghed": 2, "stolt": 2,
          "dårlig": -2, "værre": -2, "vørst": -3, "katastrofe": -3,
          "løgn": -3, "svigt": -2, "trussel": -2, "farlig": -2,
          "krise": -2, "vold": -3, "frygt": -2, "svag": -2}

ord = (udsnit["tekst"].astype(str)
      .str.lower()
      .str.replace(r"^\wæøå ", " ", regex=True)
      .str.split()
      .explode())

udsnit["score"] = ord.map(ordbog).groupby(level=0).sum()
udsnit["uden_godt"] = ord.map(ordbog).where(ord !=
"godt").groupby(level=0).sum()
```

```
print(udsnit[["sentiment", "score",
"uden_godt"]].corr(method="spearman").round(2))
```

	sentiment	score	uden_godt
sentiment	1.00	-0.27	-0.12
score	-0.27	1.00	0.90
uden_godt	-0.12	0.90	1.00

Kig efter i viewer: Sortér udsnit efter sentiment. Læs begyndelsen af den mest negative og den mest positive debat.

Til diskussion:

- Hvor mange debatter er der per område i udsnittet? Kan I sige noget om forskelle mellem områderne?
- Hvor enige er asent og ordbogen? Hvad betyder det, at to mål for *tone* kun hænger delvist sammen?
- Hvilket af de to mål ville I stole mest på? Hvorfor?

Jeres svar:

3. Hvilke ord driver tonen?

Med ordbogen fra slides var *godt* det ord, der fyldte mest i scoren (*Hvad fylder i scoren?*). Hvilke ord driver asent? Det er samme spørgsmål, nu for en sprogmodel.

Opgave:

1. Skriv en funktion `polaritetstabel(doc)`, der laver en tabel med de ord i et Doc, der har en polaritet, og deres polaritet.
2. Brug den på alle debatterne i `udsnit`, og sæt tabellerne sammen.
3. Summér polariteten for hvert ord. Hvilke ord trækker mest op, og hvilke trækker mest ned?
4. Hvor stor en andel af den samlede positive polaritet kommer fra *tak* og *godt*?

Hjælp:

- spaCy-notebooken, trin 1 og 3.3: `tokentabel()` og `pd.concat(tabeller.to_dict(), names=[...]).reset_index()`.
- Slides, *Hvorfor fik den den score?*: `tok._.polarity.polarity`.
- Preprocess-notebooken, trin 5.2: `.agg(antal="size", sum="sum")`.

```
# Din kode her
def polaritetstabel(doc):
    ramt = [t for t in doc if t._.polarity.polarity != 0]
    return pd.DataFrame({
        "ord": [t.text.lower() for t in ramt],
        "polaritet": [t._.polarity.polarity for t in ramt],
    })
```

```
tabeller = udsnit["doc"].map(polaritetstabel)

polariteter = pd.concat(tabeller.to_dict(), names=["debat",
"nr"]).reset_index()

print(len(polariteter), "ord med polaritet")
```

35468 ord med polaritet

```
bidrag = (polariteter
    .groupby("ord")["polaritet"]
    .agg(antal="size", sum="sum")
    .sort_values("sum"))

print(bidrag.head(10).round(1))
print()
print(bidrag.tail(10).round(1))
```

ord	antal	sum
svært	246	-478.4
problem	268	-439.7
desværre	219	-434.5
fattigdom	198	-369.0
kriminalitet	119	-326.2
imod	369	-310.4
problemer	206	-309.3
risiko	143	-270.6
fattige	130	-250.4
krav	246	-230.9

ord	antal	sum
sikre	644	626.1
støtte	427	639.1
ønsker	478	824.3
støtter	454	880.2
bedre	556	1044.4
mulighed	769	1334.0
god	485	1375.5
når	2281	2229.1
godt	1690	4819.1
tak	2438	4859.2

```
positiv = bidrag.loc[bidrag["sum"] > 0, "sum"]

print("Andel fra 'tak':", round(positiv["tak"] / positiv.sum(), 3))
print("Andel fra 'godt':", round(positiv["godt"] / positiv.sum(), 3))
print("Andel fra de 5 største:", round(positiv.nlargest(5).sum() /
positiv.sum(), 3))
```

```
Andel fra 'tak': 0.119
Andel fra 'godt': 0.118
Andel fra de 5 største: 0.358
```

Kig efter i viewer: Åbn bidrag. Sortér efter sum. Læs de 20 øverste og de 20 nederste.

Til diskussion:

- Hvilke af de mest positive ord er vurderinger, og hvilke er høflighed eller procedure?
- Hvilke af de mest negative ord beskriver et problem, som debatten handler om, snarere end en negativ holdning?
- Er der ord, der får en polaritet, som ikke giver mening?

Jeres svar:

Hvad har I fundet?

Udfyld tabellen, når I er færdige. Den er en huskeliste til metodeafsnittet.

Øvelse	Metode	Hvad fandt I?	Hvad skal man være opmærksom på?
1	asent på testsætninger		
2	tonen i debatterne, sammenlignet med ordbogen		
3	ord, der driver tonen		
4	høflighed og minimumsvalidering		